

```
In [1]: # Breakout Strategy

# ABDALLAH ABDELKARIM LE 2 JUILLET 2024
```

```
In [2]: #Importing the necessary libraries and defining 5Y Portfolio 3 (10 stocks)
```

```
# Import necessary libraries
import yfinance as yf
import pandas as pd
from datetime import datetime, timedelta
import numpy as np
import json
import plotly.graph_objects as go
import matplotlib.pyplot as plt

ticker_list = ['AAPL', 'AMZN', 'MSFT', 'GOOGL', 'META', 'TSLA', 'NVDA', 'ADBE', 'NFLX', 'INTC']
years = 5
```

/Users/abdelkarimabdallah/anaconda3/lib/python3.11/site-packages/yfinance/base.py:48: FutureWarning: The default dtype for empty Series will be 'object' instead of 'float64' in a future version. Specify a dtype explicitly to silence this warning.

```
_empty_series = pd.Series()
```

```
In [3]: #Part 1: Fetching Stock Data
```

```
def fetch_stock_data(ticker_list, years=5):
    end_date = datetime.now()
    start_date = end_date - timedelta(days=years * 365)

    close_data_df = pd.DataFrame()
    high_data_df = pd.DataFrame()
    low_data_df = pd.DataFrame()

    for ticker in ticker_list:
        stock = yf.Ticker(ticker)

        hist_data = stock.history(period='1d', start=start_date, end=end_date)

        close_data = hist_data['Close'].rename(ticker)
        close_data_df = pd.merge(close_data_df, pd.DataFrame(close_data), left_index=True, right_index=True, how='outer')

        high_data = hist_data['High'].rename(ticker)
        high_data_df = pd.merge(high_data_df, pd.DataFrame(high_data), left_index=True, right_index=True, how='outer')

        low_data = hist_data['Low'].rename(ticker)
        low_data_df = pd.merge(low_data_df, pd.DataFrame(low_data), left_index=True, right_index=True, how='outer')

    return close_data_df, high_data_df, low_data_df

close, high, low = fetch_stock_data(ticker_list, years)
```

```
In [4]: #Part 2: Compute the Highs and Lows in a Window
```

```
def get_high_lows_lookback(high, low, lookback_days):
    lookback_high = high.shift(1).rolling(lookback_days).max()
    lookback_low = low.shift(1).rolling(lookback_days).min()

    return lookback_high, lookback_low

lookback_days = 50
lookback_high, lookback_low = get_high_lows_lookback(high, low, lookback_days)
```

```
In [5]: #Part 3: Compute Long and Short Signals
```

```
def get_long_short(close, lookback_high, lookback_low):
    long_signal = (close - lookback_high > 0).astype('int')
    short_signal = -(close - lookback_low < 0).astype('int')
    long_short = short_signal + long_signal

    return long_short

signal = get_long_short(close, lookback_high, lookback_low)
```

```
In [6]: #Part 4: Filter Signal
```

```
def clear_signals(signals, window_size):
    clean_signals = [0]*window_size

    for signal_i, current_signal in enumerate(signals):
        has_past_signal = bool(sum(clean_signals[signal_i:signal_i+window_size]))
        clean_signals.append(not has_past_signal and current_signal)
```

```

clean_signals = clean_signals[window_size:]

return pd.Series(np.array(clean_signals).astype(int), signals.index)

def filter_signals(signal, lookahead_days):

    long_signals = (signal > 0).astype('int')
    short_signals = -(signal < 0).astype('int')

    long_signals = long_signals.apply(lambda s: clear_signals(s, window_size = lookahead_days))
    short_signals = short_signals.apply(lambda s: clear_signals(s, window_size = lookahead_days))

    filtered_signal = long_signals + short_signals

    return filtered_signal

signal_5 = filter_signals(signal, 5)
signal_10 = filter_signals(signal, 10)
signal_20 = filter_signals(signal, 20)

```

In [7]: #Part 5: Lookahead Close Prices & Price Returns

```

def get_lookahead_prices(close, lookahead_days):
    lookahead_prices = close.shift(-lookahead_days)

    return lookahead_prices

lookahead_5 = get_lookahead_prices(close, 5)
lookahead_10 = get_lookahead_prices(close, 10)
lookahead_20 = get_lookahead_prices(close, 20)

def get_return_lookahead(close, lookahead_prices):
    lookahead_returns = np.log(lookahead_prices/close)

    return lookahead_returns

price_return_5 = get_return_lookahead(close, lookahead_5)
price_return_10 = get_return_lookahead(close, lookahead_10)
price_return_20 = get_return_lookahead(close, lookahead_20)

```

In [8]: #Part 6: Compute the Signal Return

```

def get_signal_return(signal, lookahead_returns):
    signal_return = signal * lookahead_returns

    return signal_return

signal_return_5 = get_signal_return(signal_5, price_return_5)
signal_return_10 = get_signal_return(signal_10, price_return_10)
signal_return_20 = get_signal_return(signal_20, price_return_20)

```

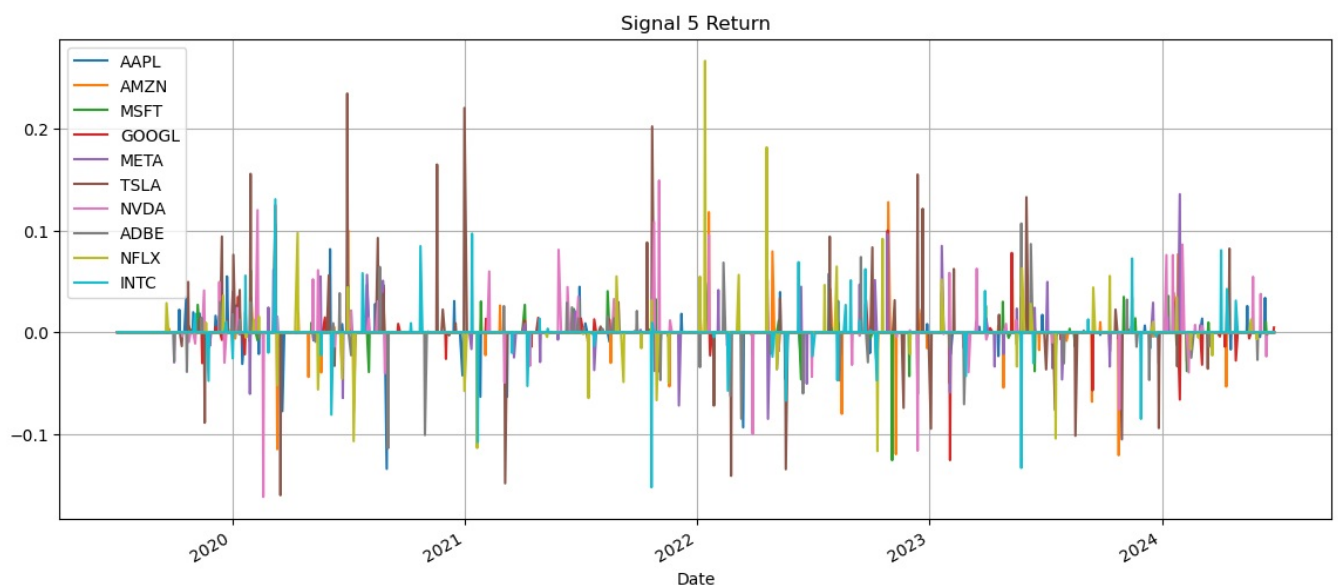
In [9]: #Part Plotting Signal 5 Return

```

import matplotlib.pyplot as plt
signal_return_5.plot(figsize=(14,6))
plt.grid()
plt.title("Signal 5 Return")

```

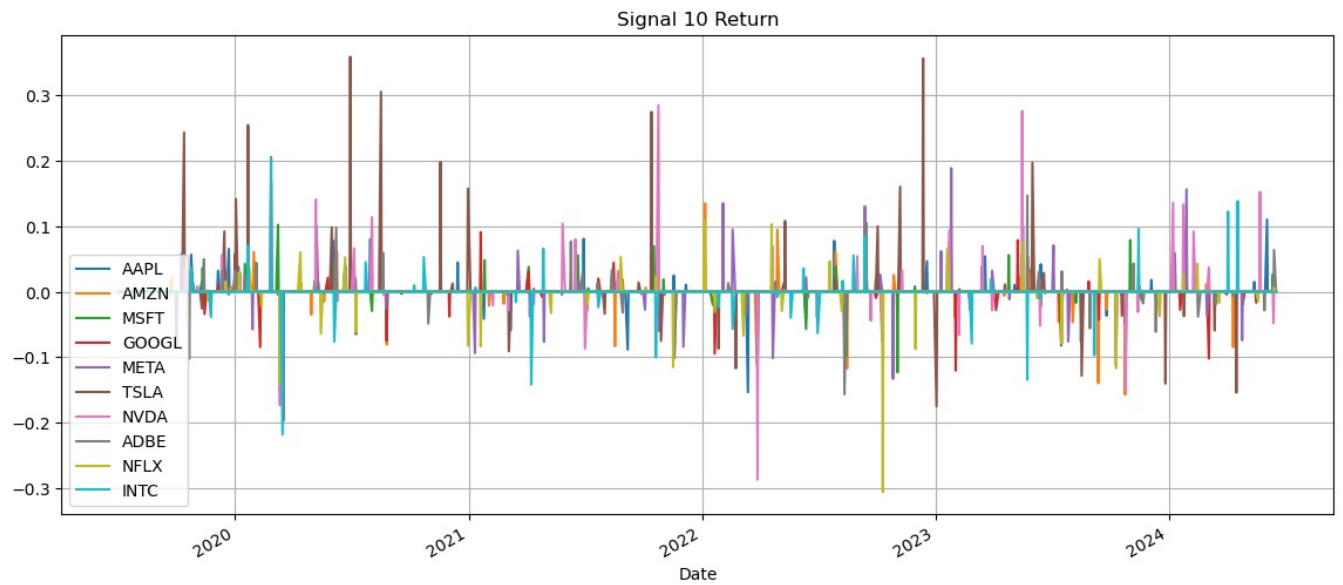
Out[9]: Text(0.5, 1.0, 'Signal 5 Return')



In [10]: # Plotting Signal 10 Return

```
signal_return_10.plot(figsize=(14,6))
plt.grid()
plt.legend(loc="lower left")
plt.title("Signal 10 Return")
```

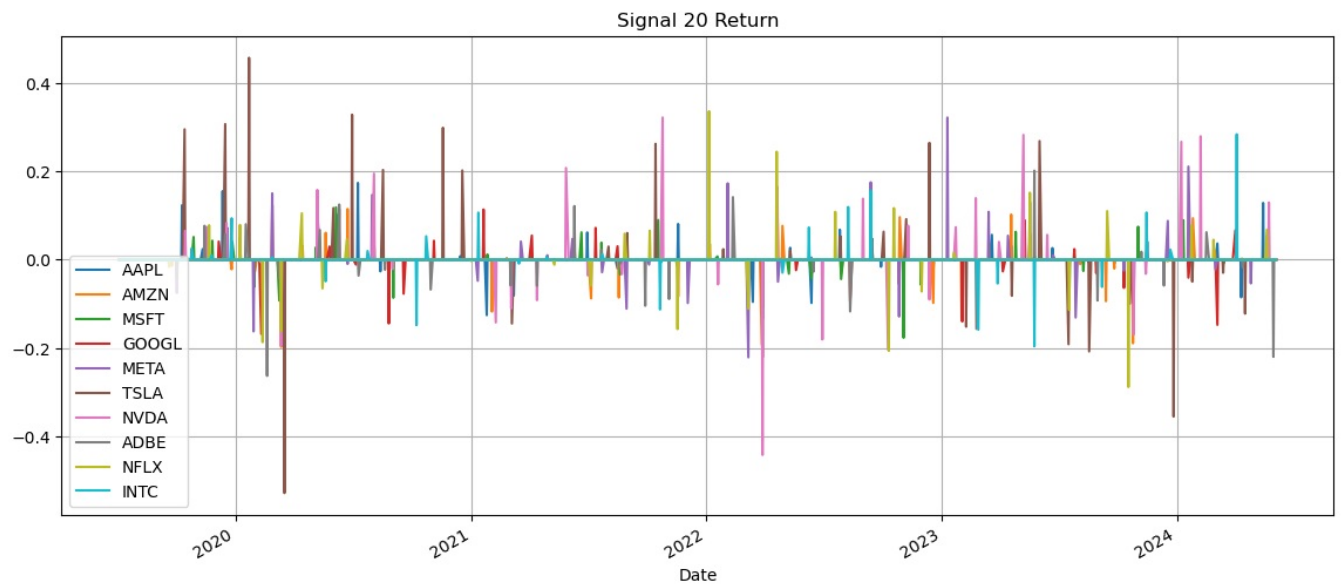
Out[10]: Text(0.5, 1.0, 'Signal 10 Return')



In [11]: # Plotting Signal 20 Return

```
signal_return_20.plot(figsize=(14,6))
plt.grid()
plt.legend(loc="lower left")
plt.title("Signal 20 Return")
```

Out[11]: Text(0.5, 1.0, 'Signal 20 Return')



In []:

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js